PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)
100 Feet Ring Road, BSK III Stage, Bengaluru-560 085

Department of Computer Science and Engineering

Linear Algebra and Its Applications

UE24MA241B

Mini Project

Team Details

1. **Guru R Bharadwaj [PES1UG24CS177]**
2. **G.V.V Sai Aarush [PES1UG24CS176]**
3. **Harshith J [PES1UG24CS185]**
4. **Gautam Nair [PES1UG24CS169]**

Cricket Player Performance Analysis Using Linear Algebra

Problem Statement

In modern cricket analytics, evaluating player performance involves multiple factors such as batting, bowling, and fielding statistics. This project applies a structured linear algebra pipeline to analyze player data, reduce redundancy, identify dominant patterns, and predict overall performance scores. The goal is to rank players and categorize them as Batsmen, Bowlers, or All-rounders using mathematical techniques.

Dataset Description

We use a dataset of 100 cricket players. Each player is described by five numerical performance features that form the columns of our matrix:

- Runs — total runs scored
- Batting Average — runs per dismissal
- Strike Rate — runs per 100 balls
- Wickets — total wickets taken
- Economy — runs conceded per over

Each player is represented as a vector:

$player_i$ = [Runs, Avg, SR, Wickets, Economy]

Note: The dataset also contains Role, Batting Style, Bowling Style, Matches, Bowling Average, and Fielding Rating columns. These are used only for display and validation in the final output, not as part of the matrix pipeline.

Step 1: Matrix Representation

We convert the dataset into a matrix A where each row represents a player and each column represents a performance feature. This transforms raw cricket data into a mathematical object that supports linear algebra operations.

Example matrix A (first 3 rows):

Player	Runs	Batting Avg	Strike Rate	Wickets	Economy
Player 1 (Batsman)	1939	27.7	126.7	0	0.0
Player 2 (Bowler)	2100	16.5	105.7	111	6.4
Player 3 (WK)	3827	42.9	149.0	0	0.0

Topics Used:
Matrices
Systems of Linear Equations
Matrices as Linear Transformations

Step 2: Matrix Simplification (Gaussian Elimination / RREF)

We apply Gaussian Elimination to convert A into Row Reduced Echelon Form (RREF). This reveals which features are linearly independent (pivot columns) and which carry redundant information (free columns). If rank = 5, all five features contribute unique information.

Purpose:

- Identify which features are linearly independent
- Detect linear dependencies among features
- Determine the rank of the data matrix

Result (10-row submatrix):

Rank = 5 → All five features are linearly independent (pivot columns: 0, 1, 2, 3, 4)

```
from sympy import Matrix
A_sym = Matrix(A[:10].tolist())
rref_matrix, pivot_cols = A_sym.rref()
print('Pivot columns:', pivot_cols)
print('Rank:', len(pivot_cols))
```

Topics Used:
Gaussian Elimination
Row Reduced Echelon Form
LU Decomposition (optional computational method)

Step 3: Structure of the Vector Space

We study the vector spaces formed by the data matrix A. Each space reveals a different structural property of the player data.

We analyze:

- Row Space → the space of all player performance vectors
- Column Space → the space spanned by the 5 feature directions
- Null Space → hidden linear relationships among features

Output:

Rank = 5 → number of linearly independent features

Nullity = 5 − Rank = 0 → no redundant features; null space is trivial

```
import scipy.linalg as la
nullity = A.shape[1] - rank
null_space_basis = la.null_space(A)
print('Rank:', rank)
print('Nullity:', nullity)
```

Topics Used:
Vector Spaces and Subspaces
Basis and Dimension
Rank and Nullity
Column Space and Null Space

Step 4: Remove Redundancy

Using the pivot columns identified from RREF, we extract a basis of linearly independent features. These pivot columns form a basis for the column space of A, eliminating any redundant directions. In our dataset, all 5 features are independent, so the full set is retained as the basis.

Result:

- Pivot columns identified: [0, 1, 2, 3, 4] → all features independent
- Basis = {Runs, Batting Avg, Strike Rate, Wickets, Economy}
- No feature needs to be dropped

```
basis_cols = list(pivot_cols)
basis = A[:, basis_cols]
print('Independent features:', [features[i] for i in basis_cols])
```

Topics Used:
Linear Independence
Basis of a Vector Space

Step 5: Orthogonalization (Gram–Schmidt)

We apply the Gram–Schmidt process to convert the feature basis into a set of orthogonal vectors $u_1, u_2, \ldots, u_5$. Orthogonal vectors are mutually perpendicular (dot product = 0), meaning each direction captures a completely independent aspect of player performance with no overlap between them.

Interpretation of orthogonal basis vectors:

- u_1 → Primary batting direction (dominant component)
- u_2 → Bowling-batting contrast direction
- u_3 → Strike-rate/wicket efficiency direction
- u_4, u_5 → Residual independent performance directions

Verification — dot products between orthogonal vectors (all ≈ 0):

$u_1 \cdot u_2 = 0.000000$ ✓

$u_1 \cdot u_3 = 0.000000$ ✓

$u_2 \cdot u_3 = 0.000000$ ✓

```
def gram_schmidt(vectors):
    orthogonal = []
    for v in vectors:
        w = v.copy().astype(float)
        for u in orthogonal:
            w -= (np.dot(v, u) / np.dot(u, u)) * u
        if np.linalg.norm(w) > 1e-10:
            orthogonal.append(w)
    return np.array(orthogonal)

feature_vectors = A[:5]   # 5 player vectors in 5D feature space
ortho_basis = gram_schmidt(feature_vectors)
```

Topics Used:
Orthogonal Vectors
Orthogonal Bases
Gram–Schmidt Orthogonalization

Step 6: Projection

We project each player's performance vector onto the orthogonal basis. The projection scalar onto each basis vector u_i tells us how strongly that player aligns with each independent performance direction. This enables us to compare players and estimate missing performance values.

Projection formula:

$\text{proj}_u(v) = (v \cdot u / u \cdot u) \times u$

Sample output — Player 1 (Batsman):

- Projection onto u_1: 1.0000 → aligns strongly with primary batting direction
- Projection onto u_2: 0.0000 → no bowling component
- Projection onto u_3 – u_5: ≈0 → residual directions minimal

```
def project_onto_basis(v, ortho_vecs):
    scalars = []
    for u in ortho_vecs:
        scalar = np.dot(v, u) / np.dot(u, u)
        scalars.append(scalar)
    return np.array(scalars)

player1 = A[0]
proj_scalars = project_onto_basis(player1, ortho_basis)
print('Projection scalars:', proj_scalars)
```

Topics Used:
Orthogonal Projections
Projection onto Subspaces

Step 7: Prediction Using Least Squares

Because the system Ax = b is overdetermined (100 equations, 5 unknowns), an exact solution may not exist. We compute the best approximate solution using the Least Squares formula. This finds the coefficient vector x̂ that minimizes the prediction error across all players.

$$\hat{x} = (A^T A)^{-1} A^T b$$

Note: b is a proxy performance score (weighted combination of statistics used as a ground truth label). In a real system, b would be actual match ratings or expert scores.

Optimal coefficient weights learned by least squares:

- Runs: 0.4000
- Batting Avg: 0.2000
- Strike Rate: 0.1000

- Wickets: 0.2000
- Economy: −0.1000

```
b = (df['Runs']*0.4 + df['Batting_Avg']*0.2 +
    df['Strike_Rate']*0.1 + df['Wickets']*0.2 -
    df['Economy']*0.1).values

x_hat = np.linalg.inv(A.T @ A) @ A.T @ b
predicted = A @ x_hat
df['Performance_Score'] = predicted
```

Topics Used:
Least Squares Solution
Matrix Transpose and Inverse

Step 8: Pattern Discovery Using Eigenvalues

We compute eigenvalues of the true covariance matrix of the player data to identify dominant performance patterns. The covariance matrix requires mean-centering A first. Large eigenvalues correspond to the strongest common trends among players.

Covariance matrix formula:

$C = (A - \bar{A})^T (A - \bar{A}) / (n - 1)$

Note: Using A^TA directly (without centering) gives the Gram matrix, not the true covariance matrix. Mean-centering is essential.

Eigenvalue results (descending order):

- Pattern 1: λ = 1,156,754 (99.7% of total variance) → batting dominance
- Pattern 2: λ = 3,656 (0.3% of variance) → bowling vs batting contrast
- Patterns 3–5: < 0.1% each → minor residual effects

```
A_centered = A - A.mean(axis=0)        # mean-center each feature
C = (A_centered.T @ A_centered) / (n-1)  # true covariance matrix

eigenvalues, eigenvectors = np.linalg.eig(C)
sorted_idx = np.argsort(eigenvalues.real)[::-1]
eigenvalues  = eigenvalues[sorted_idx].real
eigenvectors = eigenvectors[:, sorted_idx].real
```

Topics Used:
Eigenvalues and Eigenvectors

Step 9: System Simplification

Using the top k eigenvectors, we project the data into a lower-dimensional space where each axis is a pure, independent performance pattern. This removes noise, reduces complexity, and retains the most meaningful directions. With k = 2, we retain 100% of the variance while reducing from 5 features to 2 interpretable dimensions.

Reduced representation (first 5 players):

Player	Role	Pattern 1	Pattern 2
Player 1	Batsman	318.86	52.48
Player 2	Bowler	157.12	-58.17
Player 3	Wicketkeeper	-1569.04	65.07
Player 4	Bowler	-357.98	-73.03
Player 5	Wicketkeeper	-171.12	55.87

```
k = 2
top_eigvecs = eigenvectors[:, :k]      # top 2 eigenvectors
A_reduced   = A_centered @ top_eigvecs  # project into 2D space
print('Variance retained:', 100 *
eigenvalues[:k].sum()/eigenvalues.sum(), '%')
```

Topics Used:
Diagonalization of a Matrix
Diagonalization of Symmetric Matrices

Final Output

The complete pipeline produces three meaningful outputs:

1. Top 10 Players by Predicted Performance Score:

Rank	Name	Role	Performance Score
1	Player_59	Bowler	1611.98
2	Player_24	All-rounder	1592.93
3	Player_49	Wicketkeeper	1581.59
4	Player_96	All-rounder	1577.94
5	Player_34	Wicketkeeper	1565.35
6	Player_79	Wicketkeeper	1559.43
7	Player_3	Wicketkeeper	1554.28
8	Player_31	All-rounder	1524.87
9	Player_77	All-rounder	1521.45
10	Player_35	Bowler	1514.68

2. Player Category Distribution:

- Batsman: 24 predicted | 23 actual
- Bowler: 10 predicted | 26 actual
- All-rounder: 66 predicted | 28 actual
- Wicketkeeper: 0 predicted | 23 actual (not modeled by rules)

3. Dominant Performance Patterns:

- Pattern 1 accounts for 99.7% of variance → batting volume is the primary differentiator
- Pattern 2 accounts for 0.3% → bowling vs batting contrast
- Together they explain 100.0% of all variation in the dataset

Complete Code

```
import numpy as np
import pandas as pd
from sympy import Matrix
import scipy.linalg as la

df = pd.read_csv('cricket_dataset.csv')
features = ['Runs', 'Batting_Avg', 'Strike_Rate', 'Wickets', 'Economy']
A = np.array(df[features].values, dtype=float)

# Step 1: Matrix
print('A shape:', A.shape)
print(A[:5])

# Step 2: RREF
A_sym = Matrix(A[:10].tolist())
rref_matrix, pivot_cols = A_sym.rref()
rank = len(pivot_cols)
print('Rank:', rank, '| Pivot cols:', pivot_cols)

# Step 3: Vector space
nullity = A.shape[1] - rank
null_basis = la.null_space(A)
print('Nullity:', nullity)

# Step 4: Basis
basis = A[:, list(pivot_cols)]

# Step 5: Gram-Schmidt
def gram_schmidt(vectors):
```

```
    ortho = []
    for v in vectors:
        w = v.copy().astype(float)
        for u in ortho:
            w -= (np.dot(v,u)/np.dot(u,u)) * u
        if np.linalg.norm(w) > 1e-10:
            ortho.append(w)
    return np.array(ortho)
ortho_basis = gram_schmidt(A[:5])

# Step 6: Projection
def project(v, basis):
    return [np.dot(v,u)/np.dot(u,u) for u in basis]
print('Player 1 projections:', project(A[0], ortho_basis))

# Step 7: Least squares
b = (df['Runs']*0.4 + df['Batting_Avg']*0.2 +
     df['Strike_Rate']*0.1 + df['Wickets']*0.2 -
     df['Economy']*0.1).values
x_hat = np.linalg.inv(A.T @ A) @ A.T @ b
df['Performance_Score'] = A @ x_hat

# Step 8: Eigenvalues (true covariance)
A_c = A - A.mean(axis=0)
C   = (A_c.T @ A_c) / (len(A) - 1)
vals, vecs = np.linalg.eig(C)
idx  = np.argsort(vals.real)[::-1]
vals = vals[idx].real
vecs = vecs[:, idx].real
print('Eigenvalues:', np.round(vals, 2))

# Step 9: Diagonalization / dimensionality reduction
A_red = A_c @ vecs[:, :2]
print('Reduced space (first 5 players):')
print(A_red[:5])

# Final output
print(df.sort_values('Performance_Score', ascending=False).head(10))
```